

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

EEST N.º 1 “Eduardo Ader”

Actividad: Actividad 5 — Modelado de Datos con Estructuras (**struct**) y Punteros

Grupo: RootUp/SmartPlant

Integrantes: Mikaela Batirola, Lucas Benitez, Alma Lazarte y Alma Olivola.

Proyecto: Sistema de Autorriego

Versión del manual: 1.0.0

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

1. Introducción

Este manual del programador documenta los principales conceptos utilizados en el desarrollo de la Actividad 5 de Laboratorio de Programación.

El objetivo de la actividad es utilizar estructuras (`struct`) para representar entidades que contienen diferentes tipos de datos y utilizar punteros para acceder y modificar los datos mediante el operador flecha (`->`).

El programa desarrollado permite cargar los datos de una entidad y posteriormente mostrar la información almacenada y la dirección de memoria de la estructura.

Los principales conceptos utilizados son:

- Estructuras (`struct`).
- Punteros.
- Operador flecha (`->`).
- Pasaje por dirección.
- `cin.ignore()`.
- `cin.getline()`.
- Organización de datos dentro de una entidad.

2. Objetivo del programa

El programa tiene como objetivo demostrar cómo se pueden agrupar diferentes datos relacionados mediante una estructura y cómo estos datos pueden ser modificados utilizando un puntero.

Para esta actividad, la estructura utilizada representa una entidad relacionada con el proyecto de **sistema de autorriego**.

La estructura permite almacenar:

- Un identificador.
- Un nombre o descripción.
- Una métrica de operación.

De esta manera, varios datos relacionados pueden mantenerse agrupados dentro de una misma entidad.

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

3. Estructura del proyecto

El proyecto se organiza de la siguiente manera:

SISTEMAPROYECTO/

```
|
|— .gitignore
|— LICENSE
|— README.md
|
|— Docs/
|   |— EEST1_LPR2026_ACT05_G03_Informe_v1.0.0.pdf
|   |— manuales/
|       |— manual_programador_v1.0.0.pdf
|
|— src/
|   |— main.cpp
|
|— Capturas/
|   |— ejecucion_struct.PNG
```

4. Estructuras (**struct**)

Una estructura permite agrupar diferentes variables bajo una misma entidad lógica.

Una de sus principales características es que puede contener variables de diferentes tipos de datos.

Por ejemplo, en nuestro programa se utiliza:

```
struct EntidadProyecto {
    int id;
    char nombre[50];
    float metrica;
};
```

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

5. Creación de una variable de tipo `struct`

Una vez definida la estructura, podemos crear una variable utilizando el nombre de la estructura:

```
EntidadProyecto miEntidad;
```

En nuestro programa se inicializa de la siguiente manera:

```
EntidadProyecto miEntidad = {0, "Vacio - Lucas Benitez", 0.0f};
```

Esto crea una variable llamada `miEntidad` y establece valores iniciales para sus tres miembros.

La estructura contiene inicialmente:

- `id: 0`
- `nombre: "Vacio - Lucas Benitez"`
- `metrica: 0.0`

Posteriormente estos valores son modificados mediante la función `cargarDatos()`.

6. Punteros

Un puntero es una variable que almacena la dirección de memoria de otra variable.

En este programa se utiliza un puntero hacia una estructura:

```
void cargarDatos(EntidadProyecto* ptr);
```

El `*` indica que `ptr` es un puntero a una variable de tipo `EntidadProyecto`.

El puntero permite trabajar directamente sobre la estructura original.

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

7. Pasaje por dirección

El programa utiliza pasaje por dirección para enviar a la función la dirección de la estructura.

En `main()` se realiza:

```
cargarDatos(&miEntidad);
```

El operador `&` permite obtener la dirección de memoria de `miEntidad`.

La función recibe esa dirección:

```
void cargarDatos(EntidadProyecto* ptr)
```

De esta manera, `ptr` apunta directamente a `miEntidad`.

Esto permite que la función modifique los datos originales de la estructura sin tener que crear una copia completa.

8. Operador flecha (`->`)

Cuando tenemos un puntero que apunta a una estructura, utilizamos el operador `->` para acceder a sus miembros.

Por ejemplo:

```
ptr->id
```

permite acceder al miembro `id`.

En nuestro programa se utiliza:

```
cin >> ptr->id;
```

Esto permite ingresar un valor y almacenarlo directamente en el miembro `id` de la estructura original. También se utiliza:

```
cin.getline(ptr->nombre, 50);
```

para cargar el nombre o descripción. Y:

```
cin >> ptr->metrica;
```

para cargar la métrica.

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

9. Función **cargarDatos()**

La función encargada de cargar la información es:

```
void cargarDatos(EntidadProyecto* ptr)
```

Esta función recibe un puntero a una estructura **EntidadProyecto**.

Su objetivo es solicitar al usuario los diferentes datos y almacenarlos en la estructura.

El código principal de la función es:

```
void cargarDatos(EntidadProyecto* ptr) {  
    cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;  
    cout << "=> Ingrese el ID de la entidad (entero): ";  
    cin >> ptr->id;  
    cin.ignore();  
    cout << "=> Ingrese el Nombre o Descripcion: ";  
    cin.getline(ptr->nombre, 50);  
    cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";  
    cin >> ptr->metrica;  
}
```

10. Uso de **cin.ignore()**

Después de utilizar:

```
cin >> ptr->id;
```

Queda un carácter **Enter** pendiente en el búfer de entrada.

Por este motivo, antes de utilizar **getline()** se utiliza:

```
cin.ignore();
```

Su función es descartar el carácter que quedó pendiente.

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

11. Uso de `cin.getline()`

Para ingresar el nombre o descripción se utiliza:

```
cin.getline(ptr->nombre, 50);
```

`getline()` permite leer una línea completa de texto.

En este caso:

```
ptr->nombre
```

indica dónde se almacenará el texto.

El número:

```
50
```

indica el tamaño máximo utilizado para la lectura del arreglo `nombre`.

Por ejemplo, el usuario puede ingresar:

```
Sensor de temperatura
```

y ese texto queda almacenado en:

```
ptr->nombre
```

12. Aplicación al proyecto de sistema de autorriego

La estructura desarrollada puede relacionarse con el proyecto de **sistema de autorriego**.

En este contexto, una entidad podría representar un componente del sistema, por ejemplo un sensor.

La estructura puede almacenar información como:

ID → identifica el componente

Nombre → indica qué componente es

Métrica → almacena un valor relacionado con su funcionamiento

Materia: Laboratorio de Programación (LPR)

Curso: 5.º 3.ª

Año: 2026

Por ejemplo:

ID: 25

Nombre: Sensor de humedad

Métrica: 23.5

La métrica podría utilizarse para representar un valor de funcionamiento del sistema, como una medición de humedad.

De esta forma, el uso de `struct` permite agrupar información relacionada con los componentes del sistema de autorriego.

13. Compilación y ejecución

Para compilar el programa desde la terminal se utiliza:

```
g++ src/main.cpp -o src/sistema.exe
```

Una vez compilado, se puede ejecutar mediante:

```
.\src\sistema.exe
```

14. Conclusión

En esta actividad se implementó un programa en C++ utilizando una estructura (`struct`) para agrupar diferentes tipos de datos relacionados.

También se utilizaron punteros para enviar la dirección de la estructura a una función y modificar sus datos mediante el operador flecha (`->`).

Además, se trabajó con `cin.ignore()` y `cin.getline()` para realizar correctamente la entrada de datos de texto.

Estos conceptos pueden aplicarse al proyecto de sistema de autorriego, permitiendo organizar la información de los diferentes componentes del sistema y trabajar con sus datos de manera estructurada.